Het Observer Pattern

katholieke hogeschool
associatie KU Leuven
vives

Observer Pattern
Doel

In het Observer Pattern kunnen objecten zich **abonneren** op berichten van een ander object.

Rollen in het pattern:

- **subject**: het object dat berichten stuurt (de ‘uitgever’)

- **observer**: een object dat de berichten van het subject ontvangt (de ‘abonnee’)

Observers kunnen zich **op elk moment** abonneren, of hun abonnement weer opzeggen.

katholieke hogeschool
associatie KU Leuven
vives

Observer Pattern
Wat

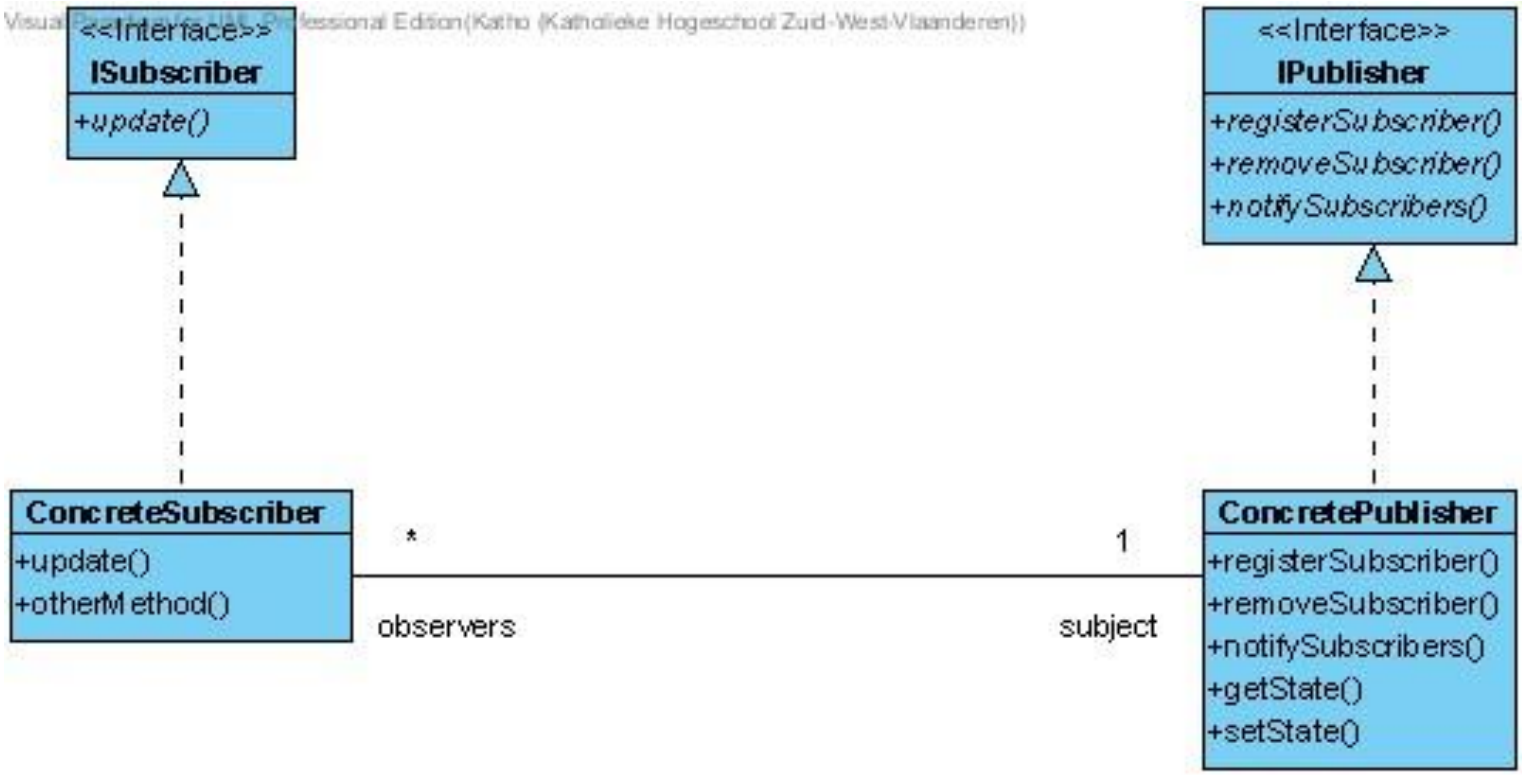
Een **één-op-veel-relatie** tussen objecten,

zodanig dat wanneer de toestand van een object verandert,

alle afhankelijke objecten **automatisch** daarvan een bericht krijgen.

katholieke hogeschool
associatie KU Leuven
vives

Observer Pattern
Structuur



katholieke hogeschool
associatie KU Leuven
vives

Observer Pattern
Push en Pull

Je kunt het observer pattern in twee stijlen implementeren.

Push

De observable stuurt uitgebreide berichten: alle nodige info zit er in.

Pull

De observable stuurt minimale berichten. Als de observer extra info wil, dan moet de observer die opvragen.

Observer Pattern
Lage Koppeling

Dit pattern is een toepassing van het **principe van lage koppeling**.

Het object dat berichten verstuurt is losgekoppeld van de ontvangers.

Dit is het tegendeel van wat je hebt bij een gewone methodeaanroep. Bij een gewone methodeaanroep is de verzender van een bericht (het object dat een methode van een ander object aanroept) heel sterk gekoppeld aan de ontvanger.

Observer Pattern
Lage Koppeling (2)

De ontkoppeling van zender en ontvanger is er door het gebruik van *interfaces*.

Een subject stuurt zijn updates naar ISubscribers, niet naar ConcreteSubscribers.

Het subject is dus niet gekoppeld aan ConcreteSubscriber.

katholieke hogeschool
associatie KU Leuven
vives

Observer Pattern in Java

In de jdk: **Observer** (interface) en **Observable** (abstracte klasse).

De jdk-klassen Observer en Observable mag je niet meer gebruiken. Die klassen zijn **deprecated**.

Er zijn een paar alternatieven:

- Implementeer alles zelf, zonder hulp van jdk-klassen.
- Gebruik EventObject en EventListener uit java.util
- Gebruik PropertyChangeListener uit java.beans
- Gebruik Flow (reactive streams - Java 9)

Observer Pattern in Java PropertyChangeListener

Publisher:

- Geen interface nodig
- Moet een veld hebben van type PropertyChangeSupport

Subscriber:

- Moet de interface PropertyChangeListener implementeren
- Die interface heeft een methode propertyChange(), met als argument een PropertyChangeEvent.
- PropertyChangeEvent heeft getters voor de gegevens van de boodschap.

Al die klassen zitten in java.beans .

katholieke hogeschool
associatie KU Leuven
vives

Observer Pattern in Java
Voorbeeld van een publisher

```
public class Publisher {
    private String news;
    private PropertyChangeSupport support;

    public Publisher() { support = new PropertyChangeSupport(this); }
    // subscriber aanmelden
    public void addPropertyChangeListener(PropertyChangeListener pcl) {
         support.addPropertyChangeListener(pcl);
    }
    // subscriber afmelden
    public void removePropertyChangeListener(PropertyChangeListener pcl) {
         support.removePropertyChangeListener(pcl);
    }
    // boodschap versturen
    public void setNews(String value) {
        support.firePropertyChange("news", this.news, value);
        this.news = value;
    }
}
```

Een naam voor de boodschap

Oude waarde

Nieuwe waarde

Observer Pattern in Java
Voorbeeld van een subscriber

```
public class Subscriber
    implements PropertyChangeListener {
    // boodschap ontvangen
    public void propertyChange(PropertyChangeEvent evt) {
        String news = (String) evt.getNewValue();
    }

    // rest van de klasse
}
```

```
// De samenwerking van publisher en subscriber
Publisher observable = new Publisher();
Subscriber observer = new Subscriber();

observable.addPropertyChangeListener(observer);
observable.setNews("news");
```

katholieke hogeschool
associatie KU Leuven
vives

Observer Pattern in .NET

Meer info: ‘Observer Design Pattern’ op msdn (http://msdn.microsoft.com/en-us/library/ee850490.aspx)

Vanaf .NET v4 .

.NET gebruikt een **push**-model.

katholieke hogeschool
associatie KU Leuven
vives

Observer Pattern in .NET

generiek

Subject: interface System.IObservable<T>

Het datatype van de gegevens die naar de observer gepusht worden

Observer: interface System.IObserver<T>

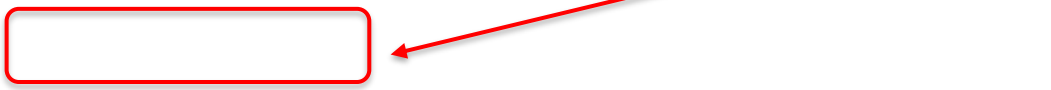
Microsoft® .NET

katholieke hogeschool associatie KU Leuven vives

Observer Pattern in .NET

IObservable<T>

De methode om een Observer toe te voegen.



Observer Pattern in .NET

Subscribe()

Subscribe() moet zelf de lijst van observers onderhouden.

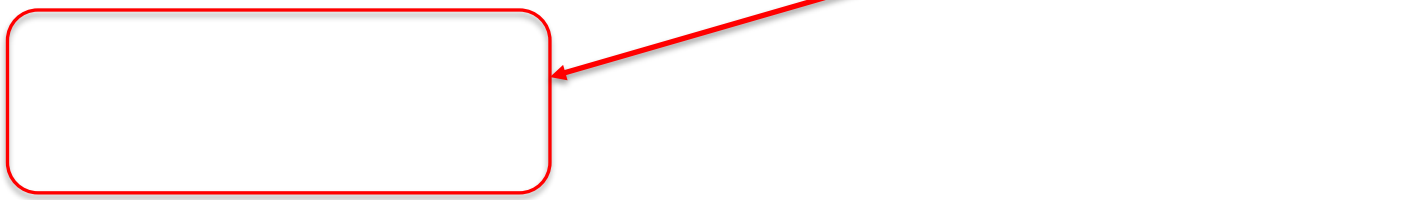
returnwaarde van Subscribe() is een object afgeleid van IDisposable: de unsubscriber.

Een observer kan de unsubscriber gebruiken om zijn abonnement stop te zetten.

(zie voorbeeldcode op msdn)

Observer Pattern in .NET

IObserver<T>



Observer Pattern in .NET

IObserver<T>: methodes

- OnNext()
  - komt overeen met update() uit het pattern
  - pusht een T-object naar de observer
- OnError()
  - signaleert de observer dat het subject het noorden kwijt is
- OnComplete()
  - signaleert de observer dat het subject geen data meer zal doorgeven

Microsoft® .NET™

katholieke hogeschool associatie KU Leuven vives

Observer Pattern
Oefeningen

- In een treinstation komen treinen aan en vertrekken ze weer.
  Een aankomst gaat over een bepaald treinnummer, een spoornummer, en een tijd van aankomst.
  Een vertrek gaat over een bepaald treinnummer, een spoornummer, en een tijd van vertrek.
  Schrijf een systeem dat het observer pattern gebruikt, om informatie te tonen over de treinen in het station.